OXIMA PLATFORM

Master Engineering & Architectural Documentation

A Complete Zero-to-Mastery Technical Manual Explaining Architecture, Frontend, Backend, Databases, Real-Time Streaming, Mathematical Formulas, and Line-by-Line Code Operations

---

Project: Oxima Enterprise Ride-Hailing Platform	**Version:** 2.4.0 (Production)
Author: Oxima Core Architecture Team	**Date:** September 2026
Target Audience: Full-Stack Engineers, DevOps, Architects	**Coverage:** 100% Codebase & Algorithms

Table of Contents Overview:

- 1. Executive System Architecture & Microservices vs Monolith Design
- 2. Frontend Engineering (React 18, Hooks, Context API, Vite, Tailwind CSS)
- 3. Backend Core (Node.js Event Loop, Express Middleware, app.js vs server.js)
- 4. Databases & Caching (MongoDB 2dsphere Geospatial & Redis GEO / 3hr TTL)
- 5. Real-Time Telemetry & Messaging (Apache Kafka vs RabbitMQ & Socket.IO 4)
- 6. Routing Algorithms, Math Formulas & Haversine Spherical Trigonometry
- 7. Line-by-Line Code Breakdown of Core Application Files
- 8. Docker Multi-Stage Containerization & CI/CD Deployment Guide

1. Executive System Architecture & Design

Oxima is built as a highly resilient, low-latency, modular distributed ride-hailing architecture designed to handle thousands of concurrent riders and drivers across all regions of India. The architecture separates high-speed geospatial read/write operations from heavy database persistence and event streaming.

Component	Technology	Architectural Responsibility
Client Layer	React 18 + Vite + Leaflet	Responsive mobile UI, map rendering, road polylines, user inputs
API Gateway / REST	Express.js 4 on Node.js	Authentication, fare calculation, ride lifecycles, validation
WebSocket Gateway	Socket.IO 4	Full-duplex live location tracking, driver presence, offers
Spatial Cache & TTL	Redis 7 (ioredis)	Sub-millisecond GEO indexing, 3-hour city surge expiration
Primary Database	MongoDB 7 (2dsphere)	ACID document persistence, historical trips, financial transactions
Event Streaming Bus	Apache Kafka 7.5	Telemetry data streaming, billing events, analytics audit log
Routing Engine	OSRM Driving API	Turn-by-turn road network calculation via Contraction Hierarchies

Modular Monolith vs Microservices:

Oxima employs a *Modular Monolith* architecture with asynchronous microservice integration (Kafka & Redis). This avoids distributed transaction overhead and network latency while allowing individual subsystems (like billing or geospatial tracking) to be extracted into standalone serverless containers whenever traffic scales.

2. Frontend Engineering: React 18, Hooks & Context API

The frontend is built with React 18 and Vite. React utilizes a Virtual DOM (VDOM) and Fiber Reconciliation to update only changed components in the DOM tree.

A. Deep Dive into React Hooks

Hook	What it does & Why it is used	Concrete Oxima Implementation
useEffect	Executes side-effects (API calls, subscriptions, timers, map events) after render. Accepts a dependency array: empty [] runs once on mount; dependencies [dep] re-run only when value changes. Return function handles cleanup.	In DriverDashboard.jsx, useEffect subscribes to Socket.IO events on mount and removes listeners on unmount to prevent memory leaks.
useState	Declares reactive state variables. When updated via setter, React schedules a re-render of the component. Preserves values across renders.	In RiderDashboard.jsx, tracks pickup, destination, activeRide, and nearbyDrivers.
useRef	Creates a mutable reference object (.current) that persists across renders without triggering a re-render when mutated. Used for DOM nodes, intervals, and simulation step trackers.	In DriverDashboard.jsx, simIntervalRef and currentSimStepRef store the simulation timer ID without causing unwanted re-renders.
useContext	Consumes values from a React Context directly without prop drilling across intermediate components.	Used via useAuth() and useSocket() to access user token, profile, and the active WebSocket instance anywhere in the app.

B. The React Context API Explained

What is Context API? It is React's built-in global state management mechanism designed to avoid 'prop drilling' (passing props through 5-10 intermediate components). It consists of a createContext() factory, a <Context.Provider value={...}>, and consumers via useContext().

Why we use it in Oxima: We use AuthContext.jsx to persist JWT tokens in localStorage, manage login/logout state, and decode user roles. We use SocketContext.jsx to establish a single, shared WebSocket connection so the entire application shares one socket connection rather than creating duplicate connections per page.

Alternatives to Context API:
- ***Redux Toolkit:*** *Best for complex state transitions with time-travel debugging (heavier boilerplate).*
- ***Zustand:*** *Lightweight hook-based store without Context Providers.*
- ***MobX:*** *Reactive observable store.*

C. Vite vs Webpack & Tailwind CSS

Why Vite 6? Webpack bundles the entire codebase before serving. Vite serves source code over native ES Modules (ESM) and leverages **esbuild** (written in Go) for pre-bundling, delivering instant start times (<100ms) and lightning-fast Hot Module Replacement (HMR).
Tailwind CSS: Uses a Just-In-Time (JIT) compiler to generate exact CSS rules on demand, keeping the production CSS bundle under 40 KB.

3. Backend Engineering: Node.js & Express Architecture

What is Node.js? Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It uses an asynchronous, single-threaded Event Loop backed by **libuv** for non-blocking I/O operations (file system, network sockets, database queries).

A. app.js vs server.js: Architectural Separation of Concerns

In production enterprise systems, app.js and server.js must be strictly decoupled:
• **app.js:** Configures Express application logic, security headers, middleware chains, route definitions, and the 404/global error handlers. It does *not* listen to a network port. This allows automated unit & integration testing with Supertest without port collisions.
• **server.js:** The entry point responsible for environment bootstrapping: connecting to MongoDB, Redis, and Kafka, instantiating the HTTP server, attaching WebSockets (Socket.IO), and listening on the designated PORT with graceful shutdown listeners.

B. The Express Middleware Pipeline

A middleware is a function with access to the Request object (req), Response object (res), and the next() function in the cycle. Oxima executes the following pipeline in exact order:

Middleware	Functionality & Security Purpose
helmet()	Sets secure HTTP response headers (HSTS, CSP, X-Frame-Options) to protect against XSS and clickjacking.
cors()	Cross-Origin Resource Sharing validator allowing the Vite frontend origin while blocking unauthorized origins.
express.json()	Parses incoming JSON request bodies with payload size limits (e.g. 10kb) to prevent DoS attacks.
rateLimit()	IP-based rate limiter (e.g. max 100 requests per 15 minutes) preventing brute force and DDoS flooding.
protect	Custom JWT middleware: extracts Bearer token, verifies cryptographic signature, attaches decoded user to req.user.
restrictTo('admin')	Role-Based Access Control (RBAC): enforces that only authorized roles can invoke administrative endpoints.
validate(zodSchema)	Zod runtime validation: ensures incoming body and params conform strictly to schema before controller execution.
errorHandler	Global error catcher: catches operational errors (AppError) and formats clean JSON responses.

4. Databases & Caching: MongoDB 7 & Redis 7

What is MongoDB? A NoSQL document database storing data in flexible BSON format (Binary JSON). Oxima utilizes MongoDB for core entities (Users, Drivers, Rides, Transactions).

Geospatial 2dsphere Indexes: MongoDB models coordinates as GeoJSON Point [longitude, latitude] and indexes them using a 2dsphere index on Earth's WGS84 ellipsoid. This enables $near and $geoWithin queries with spherical accuracy.

A. Redis 7: In-Memory Spatial Engine & 3-Hour City Surge TTL

What is Redis? Remote Dictionary Server (Redis) is an ultra-fast in-memory key-value data structure store executing operations with sub-millisecond latency.

Redis Command / Structure	How Oxima Uses It
GEOADD key lon lat member	Stores driver coordinates into geospatial index oxima:drivers:locations encoded as 52-bit Geohashes.
GEOSEARCH / GEORADIUS	Queries all online drivers within a 20 KM radius of pickup coordinates in <1 millisecond.
SET key val EX 10800	Sets City Surge Multiplier with an exact 3-Hour Time-To-Live (TTL = 10,800s). Reverts to 1.0x automatically on expiry.
ZREM member	Removes driver from spatial search pool immediately when driver clicks 'Go Offline' or disconnects.

Database Alternatives:
- ***PostgreSQL + PostGIS:*** *Best relational database for complex polygon spatial queries.*
- ***Memcached:*** *Pure key-value caching (lacks Redis GEO, Sets, and Pub/Sub).*
- ***Dragonfly / KeyDB:*** *Multi-threaded Redis drop-in replacements for extreme throughput.*

5. Real-Time Telemetry: Apache Kafka & Socket.IO 4

A. Apache Kafka 7.5: High-Throughput Event Streaming

What is Kafka? Apache Kafka is a distributed event store and stream-processing platform. It organizes events into partitioned, append-only commit logs called **Topics**. Producers publish telemetry events, while Consumers read them independently using Consumer Groups.

Oxima Kafka Topics:
1. oxima.driver.locations: Streams high-frequency GPS coordinate updates (latitude, longitude, heading, speed).
2. oxima.ride.status: Emits state transitions (REQUESTED, ACCEPTED, ARRIVED, IN_PROGRESS, COMPLETED, CANCELLED).
3. oxima.billing.events: Emits financial transaction logs for automated platform revenue audit.

Graceful Fallback Mode: *If Kafka is unreachable or offline in local development, Oxima automatically switches to an in-memory logger fallback without crashing the HTTP server.*

B. WebSockets & Socket.IO 4

Socket.IO provides full-duplex, bidirectional communication. Oxima uses Socket.IO **Rooms** for multi-tenant isolation:
• user:<userId>: Direct passenger notifications (ride accepted, driver arrived).
• driver:<driverId>: Incoming ride offers dispatched to specific drivers.
• ride:<rideId>: Live turn-by-turn trip tracking shared between driver and passenger.
• admin:fleet: Real-time fleet monitoring stream for the Admin operations cockpit.

6. Algorithms, Mathematical Formulas & Routing Engine

A. Haversine Great-Circle Distance Formula

Calculates the shortest distance over Earth's surface between two points (lat_1, lon_1) and (lat_2, lon_2) given radius $R = 6371$ km (or $6,371,000$ meters):

```
$$\Delta \phi = \frac{(lat_2 - lat_1) \cdot \pi}{180}, \quad \Delta \lambda = \frac{(lon_2 - lon_1) \cdot \pi}{180}$$
$$a = \sin^2\left(\frac{\Delta \phi}{2}\right) + \cos\left(\frac{lat_1 \cdot \pi}{180}\right) \cdot \cos\left(\frac{lat_2 \cdot \pi}{180}\right) \cdot \sin^2\left(\frac{\Delta \lambda}{2}\right)$$
$$c = 2 \cdot \arctan2\left(\sqrt{a}, \sqrt{1-a}\right), \quad d = R \cdot c$$
```

B. 20-Meter Proximity Arrival Algorithm

When a driver arrives at pickup, Oxima calculates $d_{\text{meters}} = \text{Haversine}(lat_{\text{driver}}, lon_{\text{driver}}, lat_{\text{pickup}}, lon_{\text{pickup}}) \times 1000$.
If $d_{\text{meters}} > 20$, the frontend disables the PIN input and displays distance remaining. Backend enforces $d_{\text{meters}} \le 50$ (allowing 30m GPS drift tolerance) before starting the ride.

C. Dynamic Fare & City Surge Formula

```
$$\text{Total Fare (■)} = \max\Big(\text{MinFare}, \;\big(\text{BaseFare} + (d_{\text{km}} \times \text{PerKmRate}) + (t_{\text{mins}} \times \text{PerMinRate})\big) \times \text{CitySurgeMultiplier}\Big)$$
```

D. OSRM Road Navigation & Contraction Hierarchies

Oxima queries the Open Source Routing Machine (OSRM) driving API (/route/v1/driving/...). OSRM uses **Contraction Hierarchies (CH)** to preprocess OpenStreetMap road networks, calculating turn-by-turn road paths in <5 milliseconds rather than traditional Dijkstra on raw graph nodes.

7. Line-by-Line Code Breakdown of Core Files

File: client/src/components/MapView.jsx

1. Leaflet Container setup with MapContainer, TileLayer (CartoDB Voyager).
2. MapControls: custom useMap() hook providing Recenter button (fits bounds to route) and Zoom +/- buttons.
3. Initial fit bounds guard (initialFitDoneRef) preserves manual user zoom and prevents unwanted view resets.
4. Marker Rendering: generates distinct SVG discs with heading rotation for Moto, Auto, Mini, Sedan, SUV, EV.
5. Polyline Layers: renders Apple Blue route for trip and amber dashed polyline for driver-to-pickup path.

File: client/src/pages/DriverDashboard.jsx

1. State declarations: driverProfile, isOnline (default false), activeRide, currentCoords, simSpeedIdx.
2. getDistanceToPickupMeters(): computes real-time distance in meters between driver GPS and pickup point.
3. SIM_SPEEDS array: controls simulation multiplier: 1x (1200ms), 2x (600ms), 5x (250ms), 10x (80ms).
4. startDriveSimulation(): advances vehicle step-by-step along OSRM road waypoints and emits updates via Socket.IO.
5. beforeunload listener: automatically sets driver status to OFFLINE if user closes the browser tab.

File: server/src/services/fareService.js

1. VEHICLE_PRICING dictionary: specifies base fare, per KM, per minute, min fare for all 6 vehicle types in INR.
2. setCitySurge(cityId, multiplier): writes JSON record with 3-hour TTL (10,800 seconds) to Redis.
3. getSurgeMultiplier(lat, lng): checks active city surge in Redis; reverts to 1.0x if expiresAt < Date.now().
4. getAllCitySurges(): aggregates active multipliers and calculates remaining time (e.g. '2h 45m remaining').

File: server/src/controllers/driverController.js

1. startRide(): validates driver proximity to pickup (<= 50m tolerance) and checks 4-digit PIN match.
2. completeRide(): transitions ride to COMPLETED, marks payment PAID (Cash/UPI), credits driver wallet (85%).
3. updateStatus(): toggles ONLINE/OFFLINE, manages Redis GEO index, and broadcasts to admin fleet room.

8. Docker Multi-Stage Containerization Guide

Multi-Stage Build Benefits:

1. *Minimal Image Footprint:* Build dependencies (Vite, Rollup, devDependencies) are discarded in Stage 1.
2. *Security Hardening:* Non-root user execution (USER node) prevents container breakout.
3. *Dumb-Init Signal Handling:* Handles PID 1 SIGTERM/SIGINT signals for graceful server shutdown.
4. *Automated Healthchecks:* Periodically pings /api/v1/health to ensure zero-downtime restarts.

Docker Compose Quickstart:

```
# 1. Build and start all 5 containers (Mongo, Redis, Kafka, Zookeeper, Oxima)
docker-compose up --build -d

# 2. Verify all services are healthy
docker-compose ps

# 3. Access Oxima at http://localhost:5000
```

End of Oxima Master Architectural Documentation